

Q1. Given an array, rearrange the elements such that $arr[0]$ should go to its sorted position. Also, all elements smaller than $arr[0]$ should go to its left, and all elements greater than $arr[0]$ should go to its right. After rearrangement, return the sorted index.

ex $\Rightarrow$

	0	1	2	3	4	5	6	7	8	9
	10	3	8	15	6	12	2	18	7	4

sort $\hookrightarrow$

	0	1	2	3	4	5	6	7	8	9
	4	2	3	6	7	8	10	12	15	18

ex $\Rightarrow$

	0	1	2	3	4	5	6	7	8	9
	3	8	6	2	7	4	10	18	12	15

$\underbrace{\hspace{15em}}_{< 10}$
 $\underbrace{\hspace{10em}}_{> 10}$

Brute force

sort the array

$$TC \Rightarrow O(N \log N)$$

Optimise

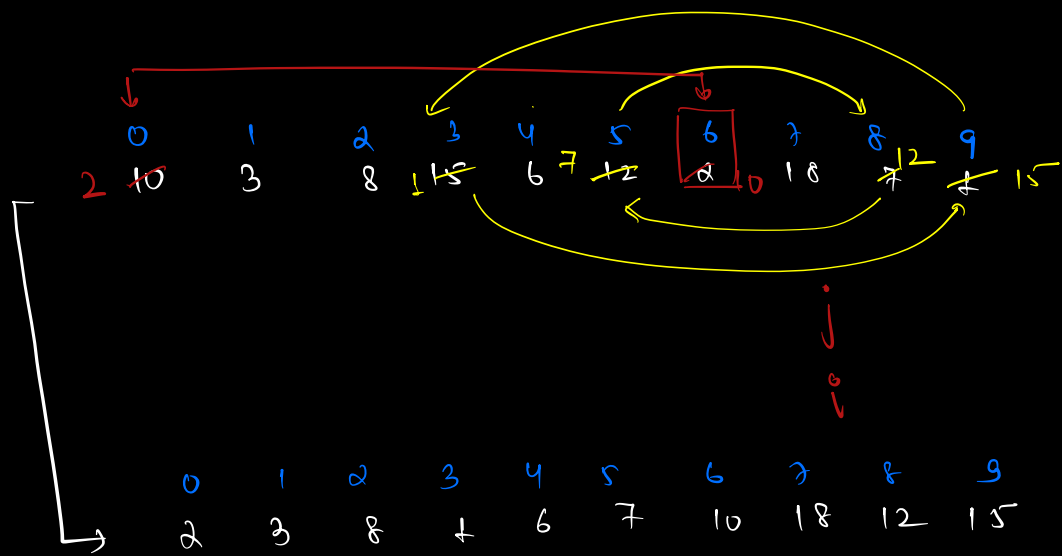
	0	1	2	3	4	5	6	7	8	9
	10	3	8	15	6	12	2	18	7	4

temp arr $\rightarrow$

	0	1	2	3	4	5	6	7	8	9
	3	8	6	2	7	4	10	18	12	15

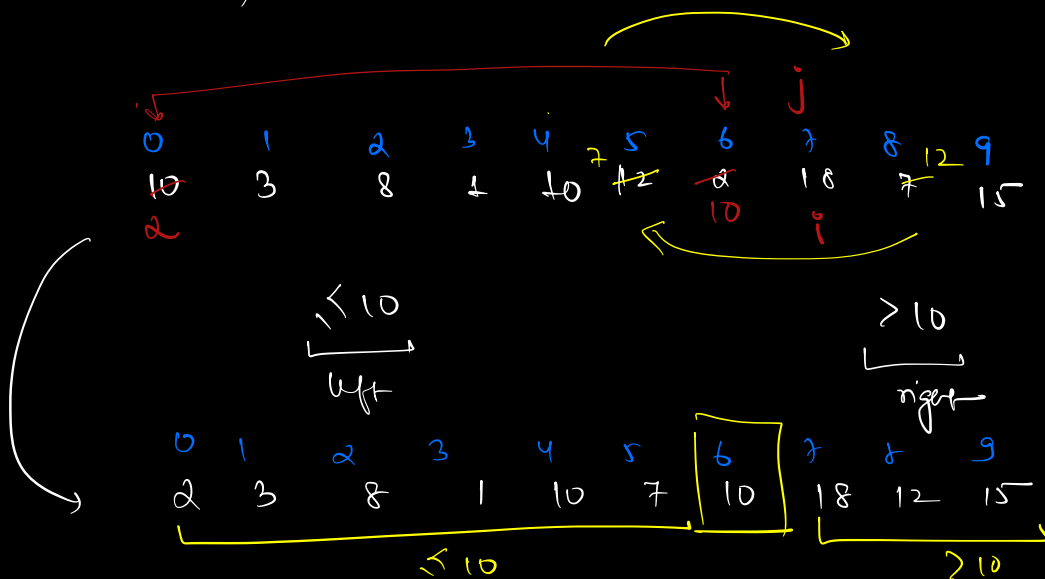
$\underbrace{\hspace{15em}}_{< 10}$
 $\underbrace{\hspace{10em}}_{> 10}$

$TC \Rightarrow O(N)$ $SC \Rightarrow O(N)$
--



- i if i is happy $\rightarrow i++$
- ii if i is unhappy but j is happy $\rightarrow j--$
- iii if i & j are unhappy $\rightarrow$ swap
 $i++$
 $j--$

TC $\Rightarrow O(N)$
 SC $\Rightarrow O(1)$

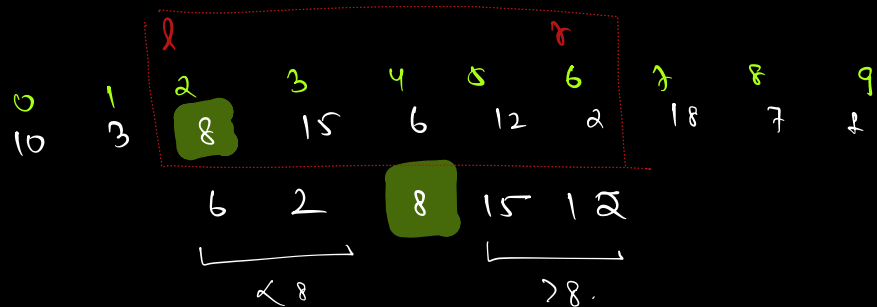


```

int rearrange ( arr[] ) {
    i = 1
    j = N-1;
    while ( i < j ) {
        if ( arr[i] <= arr[0] ) {
            i++
        } else if ( arr[j] > arr[0] ) {
            j--
        } else {
            swap ( arr[i], arr[j] )
            i++
            j--
        }
    }
    swap ( arr[0], arr[i-1] )
    return i-1;
}

```

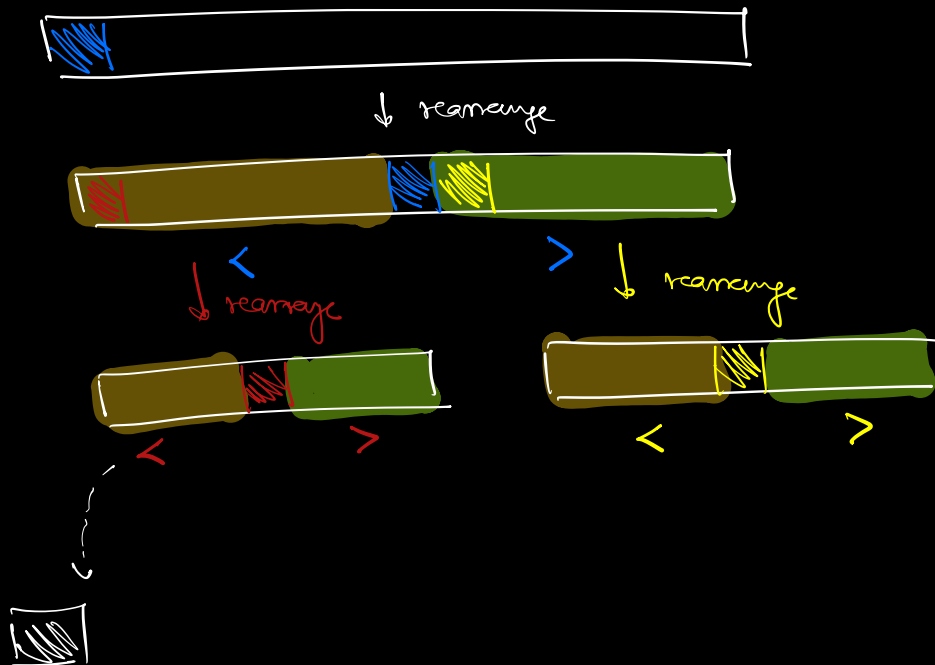
∴ recursion → $(l \rightarrow r) \rightarrow \text{rearrange}$



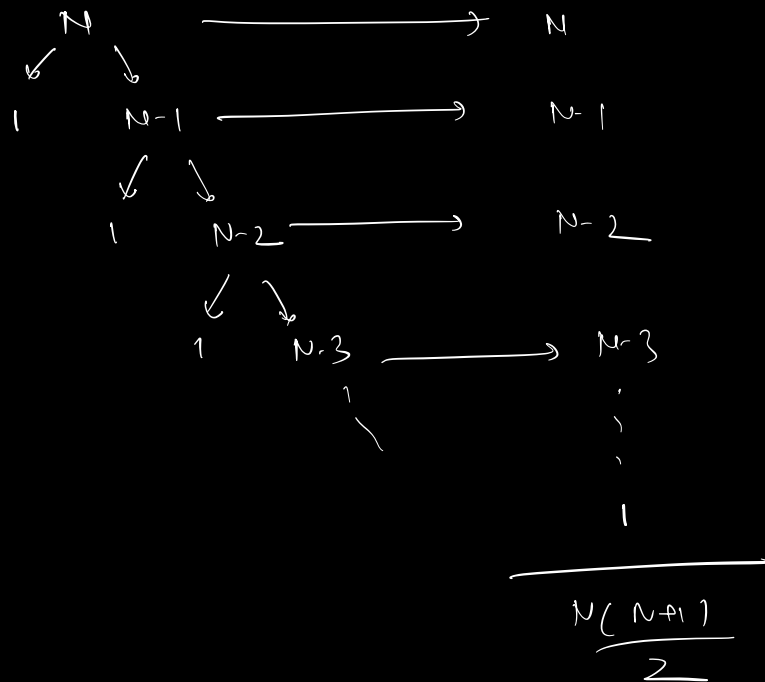
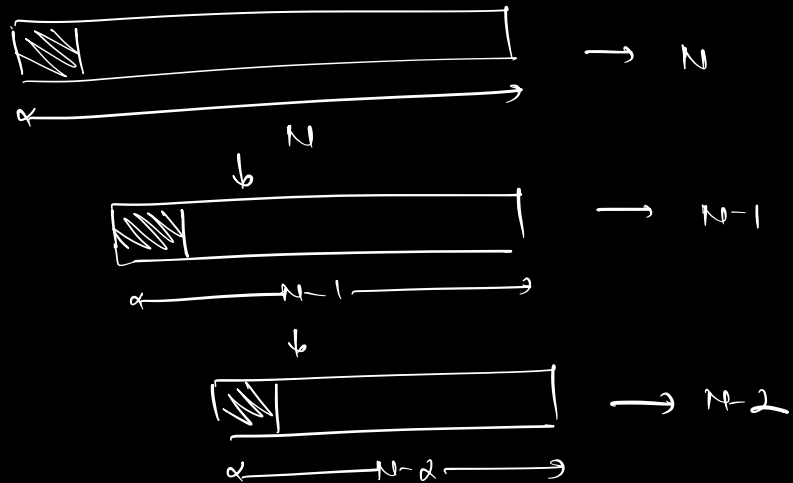
```

int rearrange ( arr[], l, r) {
    i = l+1;
    j = r;
    while ( i < j ) {
        if ( arr[i] <= arr[l] ) {
            i++;
        } else if ( arr[j] > arr[l] ) {
            j--;
        } else {
            swap ( arr[i], arr[j] );
            i++;
            j--;
        }
    }
    swap ( arr[l], arr[i-1] );
    return i-1;
}

```

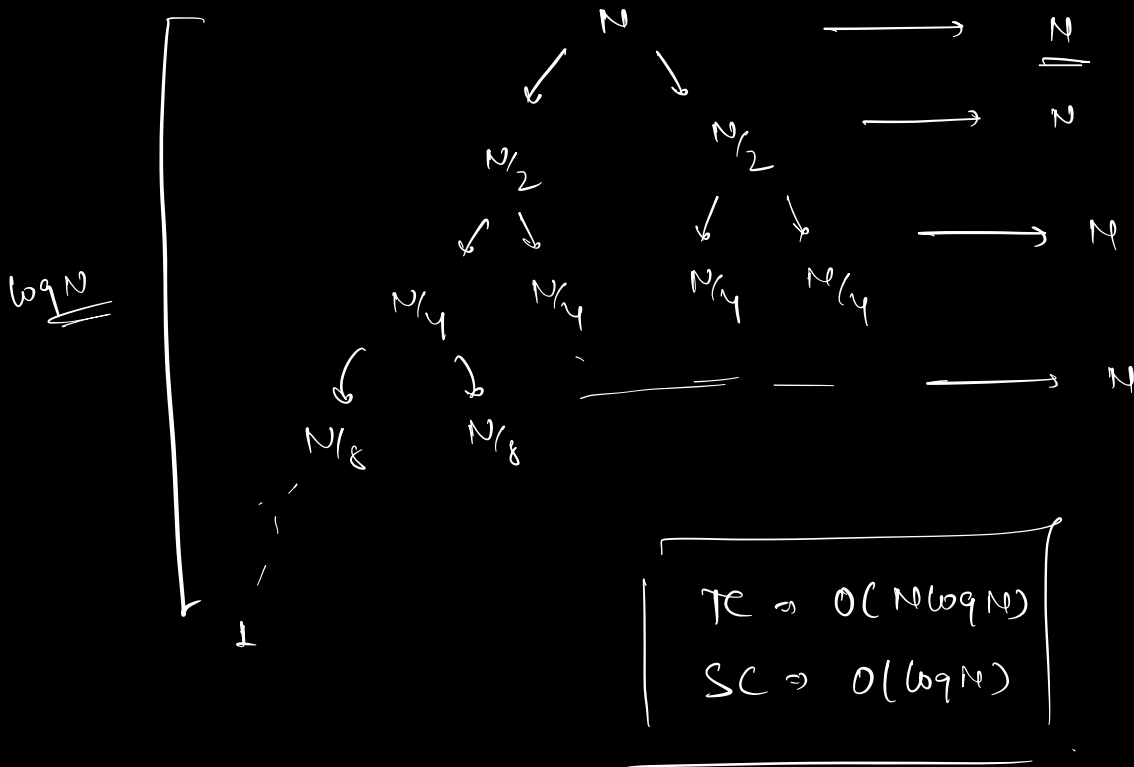
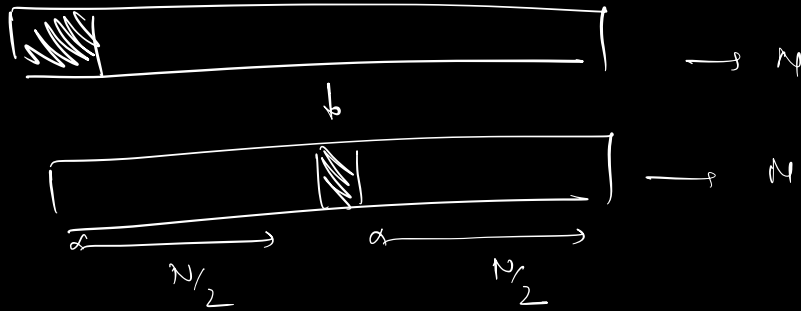


11 Analysis for quick sort:



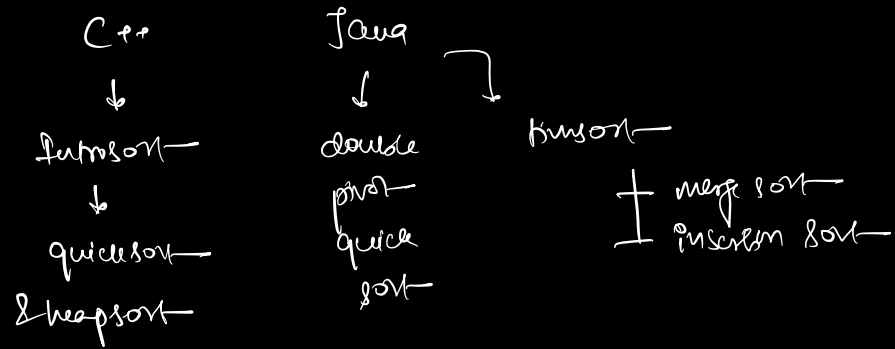
$$TC = O(N^2)$$

↑
worst case
for quick sort



1) Pivot element :-

- * first element
- * last element
- * middle element
- * random element [Randomised pivot]



Merge sort

TC $\rightarrow O(N \log N)$

SC $\rightarrow O(N)$

Quicksort

99%	99.9	1-0.1
<u>TC $\rightarrow O(N \log N)$</u>		$O(N^2)$
<u>SC $\rightarrow O(\log N)$</u>		$O(N)$

inplace	$\rightarrow$ ✓
stable	$\rightarrow$ ✗

break $\rightarrow$ 10 min

8:25 AM

Put $x =$ $\begin{matrix} 5 & 4 & 3 & 2 & 1 & 0 \\ 3 & 4 & 5 & 6 & 7 & 8 \end{matrix}$

return the digit at 3.

o/p = 5

$x_{/10} \rightarrow 3 \ 4 \ 5 \ 6 \ 7$

$x_{/100} \rightarrow 3 \ 4 \ 5 \ 6$

$x_{/1000} \rightarrow 3 \ 4 \ 5 \rightarrow \%10 \rightarrow \boxed{5}$

$\boxed{x}$ i^{th} digit $\rightarrow \left[\left(\frac{x}{10^i} \right) \% 10 \right] \rightarrow i^{\text{th}} \underline{\underline{\text{digit}}}$

array $\rightarrow$ $\begin{matrix} 361 & 432 & 12 & 78 & 500 & 112 & 02 \end{matrix}$

$\downarrow$

sort by
value of
digit on 10^i 's $\rightarrow$ $\left[500 \ 02 \ 12 \ 112 \ 432 \ 361 \ 78 \right]$

place, and

maintain
stability

$\Rightarrow$
count
sort

0	$\rightarrow$	500, 02
1	$\rightarrow$	12, 112
2	$\rightarrow$	X
3	$\rightarrow$	432
4	$\rightarrow$	X
5	$\rightarrow$	X
6	$\rightarrow$	361
7	$\rightarrow$	78

iterate [0-9] and print

[500, 2, 12, 112, 432, 361, 78]

$$TC \Rightarrow O(N+k) \rightarrow O(N+10) \sim O(N)$$

↓
10

$$SC \Rightarrow O(N)$$

Radix Sort

2 1 0
3 6 1
4 3 2
1 2
7 8
5 0 0
1 1 2
2

8th
pos

2 1 0
5 0 0
3 6 1
4 3 2
1 2
1 1 2
2
7 8

1st pos

2 1 0
5 0 0
6 0 2
0 1 2
1 1 2
4 3 2
3 6 1
0 7 8

2nd
pos

2 1 0
0 0 2
0 1 2
0 7 8
1 1 2
3 6 1
4 3 2
5 0 0

$$TC \Rightarrow O(N \times \text{max no. of digits})$$

$$SC \Rightarrow O(N)$$

$$O(N)$$

↓

$$O(N)$$

	TC	SC	Stable
<u>Selection</u>	$O(N^2)$	$O(1)$	No
<u>Bubble</u>	$O(N^2)$	$O(1)$	Yes
<u>Insertion</u>	$O(N) \rightarrow O(N^2)$	$O(1)$	Yes
<u>Merge</u>	$O(N \log N)$	$O(N)$	Yes
<u>QuickSort</u>	$O(N \log N) \rightarrow O(N^2)$	$O(1)$	No.
<u>Count</u>	$O(N+k)$	$O(k)$	Yes $\rightarrow$ possible
<u>Radix</u>	$O(n \cdot \text{digits})$	$O(N)$	Yes.

